

NPC Class Template System — Master Implementation Plan

Document Version: 1.0
Date: 2026-05-25
Status: Implementation-Ready
Companion Documents: npc_class_definitions.md , creature_class_application_system.md , npc_templates_by_level.md

Table of Contents

- [1. Executive Summary](#)
 - [2. System Architecture Overview](#)
 - [3. Integration Points with Existing Systems](#)
 - [4. Implementation Phases](#)
 - [5. Timeline Estimates](#)
 - [6. Technical Requirements](#)
 - [7. Risk Assessment & Mitigations](#)
 - [8. Testing Requirements](#)
 - [9. Success Criteria](#)
-

1. Executive Summary

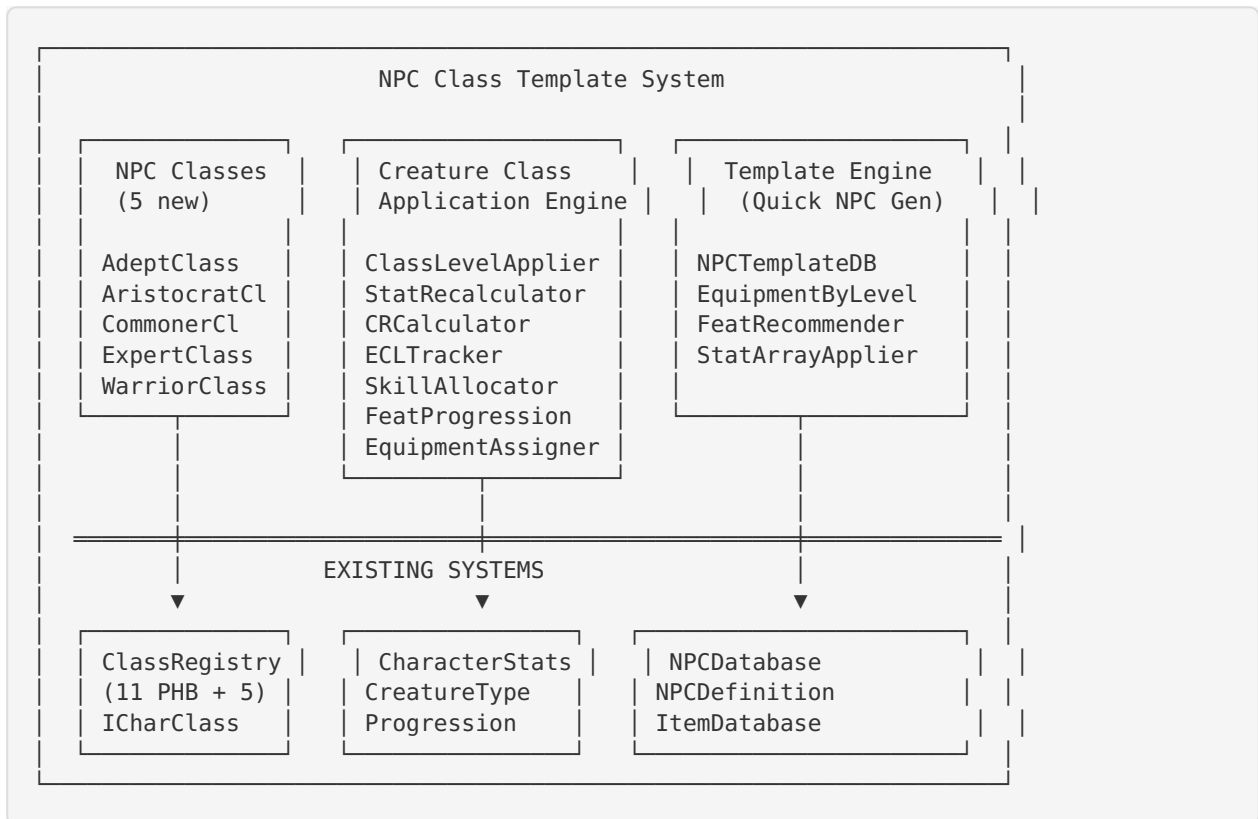
This plan covers the implementation of the **NPC Class Template System** for the D&D 3.5e Unity prototype. The system adds two major capabilities:

1. **Five NPC Classes** (Adept, Aristocrat, Commoner, Expert, Warrior) — lightweight character classes for the general populace, integrated into the existing `ICharacterClass` / `ClassRegistry` architecture.
2. **Creature Class Application Engine** — a system that applies any class levels (PC or NPC) to any creature, automatically recalculating HD, BAB, saves, skills, feats, ability scores, CR, ECL, and equipment.

The existing codebase already provides strong foundations: 11 PHB classes registered in `ClassRegistry`, a `CreatureTypeProgression` system with all 15 creature types, multiclass support via `ClassLevelEntry`, and a comprehensive `NPCDatabase` with 100+ monster definitions. This plan builds directly on these systems.

2. System Architecture Overview

2.1 High-Level Component Diagram



2.2 New File Structure

```
Assets/Scripts/
├── Classes/
│   ├── NPC/
│   │   ├── AdeptClass.cs           # Adept ICharacterClass
│   │   ├── AristocratClass.cs      # Aristocrat ICharacterClass
│   │   ├── CommonerClass.cs        # Commoner ICharacterClass
│   │   ├── ExpertClass.cs          # Expert ICharacterClass
│   │   ├── WarriorClass.cs         # Warrior ICharacterClass
│   │   ├── AdeptSpellList.cs       # Adept spell database
│   │   └── ClassRegistry.cs        # Modified: +5 NPC registrations
│   └── Character/
│       ├── CreatureClassEngine/
│       │   ├── ClassLevelApplier.cs # Core engine: apply class to creature
│       │   ├── CreatureStatRecalculator.cs # Recalc all stats after class add
│       │   ├── CRCalculator.cs       # CR adjustment formulas
│       │   ├── ECLTracker.cs         # ECL & Level Adjustment tracking
│       │   ├── CreatureSkillAllocator.cs # Skill point distribution
│       │   ├── CreatureFeatProgression.cs # Feat grants by total HD
│       │   ├── AbilityScoreProgression.cs # +1 ability every 4 total HD
│       │   └── StatArrayApplier.cs   # Elite/Nonelite/Basic arrays
│       ├── CharacterStats.cs        # Modified: new fields for RHD, LA
│       └── NPCEquipment/
│           ├── EquipmentByLevel.cs  # Wealth & gear tables
│           └── NPCEquipmentAssigner.cs # Auto-equip by class & level
└── Templates/
    ├── NPCTemplateDatabase.cs       # Quick-gen template lookup
    ├── NPCTemplate.cs               # Template data structure
    └── ExampleClassedCreatures.cs    # Pre-built monster+class combos
```

3. Integration Points with Existing Systems

3.1 ClassRegistry (Direct Extension)

The 5 NPC classes implement `ICharacterClass` and register into `ClassRegistry.Init()`. No interface changes required.

```
// ClassRegistry.cs – additions to Init()
Register(new AdeptClass());
Register(new AristocratClass());
Register(new CommonerClass());
Register(new ExpertClass());
Register(new WarriorClass());
```

Impact: All existing code that calls `ClassRegistry.GetClass(name)` automatically supports NPC classes. `CharacterStats.ClassLevels` can already hold any class name.

3.2 CharacterStats (Minor Additions)

New fields needed on `CharacterStats` :

```
// New fields for creature class application
public int RacialHitDice;           // Monster's base HD count
public int LevelAdjustment;         // LA for ECL calculation
public string AssociatedClassHint;   // "Fighter,Barbarian" for CR calc
public bool IsNPCClass;             // Quick flag for NPC vs PC class
```

These are additive — no existing fields change.

3.3 CreatureTypeProgression (Already Complete)

The existing `CreatureTypeProgressionDatabase` already has all 15 creature types with their HD, BAB, and save progressions. The `ProgressionCalculator` class provides `CalculateBAB()`, `CalculateSave()`, and `CalculateAverageHpFromHitDice()`. No changes needed.

3.4 NPCDatabase / NPCDefinition (Extension)

NPCDefinitions already store creature type, HD, and combat stats. New optional fields:

```
// NPCDefinition additions
public int RacialHitDice;           // e.g., 4 for Ogre
public int LevelAdjustment;         // e.g., +2 for Drow
public string[] AssociatedClasses;   // e.g., {"Fighter", "Barbarian"} for
Ogre
public string AdvancementType;      // "ByCharacterClass", "ByHD", "None"
```

3.5 Existing Equipment System

`ItemDatabase`, `InventoryComponent`, and `CharacterEquipment` are already functional. The new `NPCEquipmentAssigner` will use `ItemDatabase.CloneItem()` and `InventoryComponent.DirectEquip()` — the same patterns used by `FighterClass.SetupStartingEquipment()`.

3.6 FeatManager / FeatDefinitions

Existing feat system supports `HasFeat()`, `AddFeat()`, and prerequisite checking. The `CreatureFeatProgression` module will use these APIs to grant feats at the correct total HD thresholds.

3.7 Spellcasting Systems

The existing `SpellDatabase`, `SpellSlotManager`, and per-class spell lists (Cleric, Wizard, Sorcerer, Bard, Druid, Ranger, Paladin) are already implemented. The Adept class requires a new spell list (`AdeptSpellList.cs`) but uses the same infrastructure.

4. Implementation Phases

Phase 1: NPC Class Definitions (Foundation)

Scope: Create the 5 NPC class files implementing `ICharacterClass`, register them in `ClassRegistry`.

Deliverables:

- `AdeptClass.cs` — d6 HD, Poor BAB, Good Fort/Will, 2+Int skills, divine spellcasting, familiar at 2nd
- `AristocratClass.cs` — d8 HD, Medium BAB, Good Will, 4+Int skills, martial weapons, all armor/shields
- `CommonerClass.cs` — d4 HD, Poor BAB, no good saves, 2+Int skills, one simple weapon only
- `ExpertClass.cs` — d6 HD, Medium BAB, Good Reflex, 6+Int skills, 10 chosen class skills

- `WarriorClass.cs` — d8 HD, Good BAB, Good Fort, 2+Int skills, martial weapons, all armor/shields
- `AdeptSpellList.cs` — Complete 0th–5th level spell list for Adept
- Updated `ClassRegistry.cs` with 5 new registrations

Dependencies: None (builds on existing interface)

Phase 2: CharacterStats Extensions

Scope: Add fields and methods to `CharacterStats` for tracking racial HD, level adjustment, and NPC class metadata.

Deliverables:

- New fields: `RacialHitDice`, `LevelAdjustment`, `AssociatedClassHint`, stat array type
- `TotalHitDice` property: `RacialHitDice + sum(ClassLevels.Level)`
- `EffectiveCharacterLevel` property: `TotalHitDice + LevelAdjustment`
- Modified `RecalculateAllStats()` to incorporate racial HD into BAB/saves
- Backward compatibility: all new fields default to 0/null, existing creatures unaffected

Dependencies: Phase 1

Phase 3: CR & ECL Calculation Engine

Scope: Build the mathematical engine for Challenge Rating and Effective Character Level.

Deliverables:

- `CRCalculator.cs` — Static methods for:
 - `CalculateCR(baseCR, classLevels, associatedClasses, racialHD)` — full CR formula
 - Associated class: +1 CR per level
 - Nonassociated class: +½ CR per level (up to RHD), then +1 per level
 - NPC classes always nonassociated
 - Fractional CR rounding rules
- `ECLTracker.cs` — `CalculateECL(racialHD, classLevels, levelAdjustment)`
- Unit tests covering all edge cases (Ogre Barbarian, Troll Wizard, Kobold Sorcerer, etc.)

Dependencies: Phase 2

Phase 4: Creature Class Application Engine

Scope: Build the core engine that takes a base creature and adds class levels, recalculating all stats.

Deliverables:

- `ClassLevelApplier.cs` — Main entry point:

```
csharp
```

```
public static CharacterStats ApplyClassLevels(
    NPCDefinition baseCreature,
    string className,
    int classLevel,
    StatArrayType statArray = StatArrayType.Elite)
```

- `CreatureStatRecalculator.cs` — Recalculates:
 - Total HD (racial + class)
 - Combined BAB (racial BAB + class BAB)
 - Combined saves (racial + class, stacking)
 - HP (racial HP + class HP using appropriate hit dice)
 - Ability scores (racial base + array assignment + racial modifiers + level increases)
- `CreatureSkillAllocator.cs` — Allocates skill points from class levels, respecting max ranks =

TotalHD + 3

- `CreatureFeatProgression.cs` — Grants feats at 1st HD, then every 3 HD
- `AbilityScoreProgression.cs` — +1 to an ability at every 4 total HD
- `StatArrayApplier.cs` — Applies Elite/Nonelite/Basic arrays to creatures

Dependencies: Phase 3

Phase 5: Equipment & Wealth System

Scope: Build the NPC equipment assignment system based on level, class, and wealth tables.

Deliverables:

- `EquipmentByLevel.cs` — Data tables:
- Character Wealth by Level (levels 1–20)
- NPC Gear Value table
- Equipment loadout recommendations by class archetype and level
- `NPCEquipmentAssigner.cs` — Auto-equip logic:
- Select appropriate weapons/armor by class proficiency
- Assign magic items within wealth budget
- Scale enhancement bonuses by level tier
- Handle creature-specific constraints (size, natural weapons)
- Integration with existing `ItemDatabase` and `InventoryComponent`

Dependencies: Phase 4

Phase 6: Template Database & Quick Generation

Scope: Create a template library for instant NPC/monster generation.

Deliverables:

- `NPCTemplate.cs` — Data class for pre-built configurations
- `NPCTemplateDatabase.cs` — Lookup tables for:
- All 11 PHB classes at levels 1, 5, 10, 15, 20
- All 5 NPC classes at levels 1, 5, 10
- Common classed creatures (Ogre Barbarian 3, Lizardfolk Druid 5, Kobold Sorcerer 4, etc.)
- `ExampleClassedCreatures.cs` — Pre-calculated stat blocks for common combos
- API: `NPCTemplateDatabase.Generate("Ogre", "Barbarian", 3)` → complete `CharacterStats`

Dependencies: Phase 5

Phase 7: Integration & Polish

Scope: Wire the system into the game's UI, encounter system, and AI.

Deliverables:

- `NPCDatabase` entries updated with `RacialHitDice`, `LevelAdjustment`, `AssociatedClasses`
- Encounter generator can spawn classed creatures
- Tooltip/UI shows class levels on creature names (e.g., "Ogre Barbarian 3")
- AI profile auto-selection for classed creatures
- Performance optimization for batch generation

Dependencies: Phase 6

Phase 8: Validation & Comprehensive Testing

Scope: End-to-end validation against SRD reference stat blocks.

Deliverables:

- Automated test suite comparing generated stat blocks to SRD examples
- Edge case testing (0 racial HD humanoids, high-LA creatures, multiclass monsters)
- Performance benchmarks for batch NPC generation
- Documentation finalization

Dependencies: Phase 7

5. Timeline Estimates

Phase	Description	Estimated Duration	Cumulative
1	NPC Class Definitions	2-3 days	2-3 days
2	CharacterStats Ex-tensions	1-2 days	3-5 days
3	CR & ECL Calculation Engine	2-3 days	5-8 days
4	Creature Class Ap-plication Engine	3-4 days	8-12 days
5	Equipment & Wealth System	2-3 days	10-15 days
6	Template Database & Quick Gen	2-3 days	12-18 days
7	Integration & Polish	2-3 days	14-21 days
8	Validation & Testing	2-3 days	16-24 days

Total Estimate: 16-24 development days (3-5 weeks)

6. Technical Requirements

6.1 Codebase Standards

- All classes implement `ICharacterClass` — no new interfaces needed
- Static utility classes for calculation engines (matches existing `ProgressionCalculator` pattern)
- Defensive null checks on all creature lookups
- `Debug.Log` for initialization, `Debug.LogWarning` for fallbacks

6.2 Data Accuracy Requirements

- All progression tables must match D&D 3.5e SRD values exactly
- CR calculations must handle fractional CRs (stored as strings: "1/2", "1/3", etc.)

- BAB progression: Good = level, Medium = $\frac{3}{4}$ level, Poor = $\frac{1}{2}$ level (integer division)
- Save progression: Good = $2 + \text{level}/2$, Poor = $\text{level}/3$ (integer division)
- Feat progression: 1st HD + every 3 HD thereafter (1, 3, 6, 9, 12, ...)
- Ability score increase: every 4 total HD (4, 8, 12, 16, 20)

6.3 Performance Requirements

- Single NPC generation: < 5ms
- Batch generation (20 NPCs): < 50ms
- No heap allocations in hot path (reuse lists/arrays where possible)

6.4 Compatibility Requirements

- All existing NPCDatabase entries continue to work without modification
- Existing CharacterStats instances with `ClassLevels` containing PHB classes are unaffected
- New fields default to zero/null, maintaining backward compatibility

7. Risk Assessment & Mitigations

Risk	Probability	Impact	Mitigation
ICharacterClass interface insufficient for NPC class features (e.g., Expert's 10 chosen skills)	Medium	Medium	Add optional <code>CustomClassSkills</code> property; Expert overrides <code>ClassSkills</code> dynamically
CR fractional math rounding errors	Medium	High	Use integer arithmetic with $\times 2$ scaling (store half-CRs as integers internally)
Adept spellcasting doesn't fit existing spell slot system	Low	High	Adept uses same divine preparation model as Cleric; create parallel spell list
Equipment auto-assignment exceeds wealth budget	Medium	Medium	Budget-first algorithm: allocate primary weapon/armor first, then fill remaining
Existing NPCDefinitions lack racial HD data	Certain	Low	Phase 7 adds the data; until then, creatures use existing stats as-is

8. Testing Requirements

8.1 Unit Tests (Per Phase)

Phase 1 — NPC Class Tests:

- Each NPC class returns correct HitDie, BABAtLevel3, SkillPointsPerLevel, save booleans
- ClassRegistry contains all 16 classes after Init()
- Adept spell list contains correct spells at each level

Phase 3 — CR Calculation Tests:

```
// Ogre (CR 3) + Barbarian 2 (associated) = CR 5
Assert.AreEqual(5f, CRCalculator.Calculate(3f, "Barbarian", 2, 4, new[]{"Fighter", "Bar-
barian"}));

// Ogre (CR 3) + Wizard 4 (nonassociated, 4 <= 4 RHD) = CR 3 + 2 = 5
Assert.AreEqual(5f, CRCalculator.Calculate(3f, "Wizard", 4, 4, new[]{"Fighter", "Bar-
barian"}));

// Troll (CR 5) + Wizard 8 (6 nonassoc @½ = +3, 2 @1 = +2) = CR 10
Assert.AreEqual(10f, CRCalculator.Calculate(5f, "Wizard", 8, 6, new[]{"Fighter", "Bar-
barian"}));
```

Phase 4 — Stat Recalculation Tests:

- Ogre + Barbarian 2: total HD = 6, BAB = 5, HP correct
- Kobold Sorcerer 4: verify 0 racial HD humanoid handled correctly
- Save stacking: racial good + class good produces correct combined value

Phase 5 — Equipment Tests:

- Level 5 Fighter NPC has gear value ≈ 9,000 gp
- Level 10 Wizard NPC includes headband of intellect
- Equipment respects class proficiency (no plate armor on Wizard)

8.2 Integration Tests

- Generate an Ogre Barbarian 3 and compare against SRD example stat block
- Generate a Human Warrior 5 and verify all values match research document
- Spawn a classed creature in combat, verify it attacks/takes damage correctly
- Multiclass monster: Drow Fighter 2/Rogue 3 with LA +2

8.3 Regression Tests

- All existing 11 PHB classes still function identically
- All existing NPCDatabase creatures spawn correctly
- `CharacterStats` serialization/deserialization handles new fields gracefully

9. Success Criteria

9.1 Functional Criteria

Criterion	Measurement
All 5 NPC classes registered and functional	<code>ClassRegistry.Count == 16</code>
Any creature can receive any class levels	<code>ClassLevelApplier.Apply()</code> succeeds for all creature type + class combos
CR calculation matches SRD rules	100% of test vectors pass
ECL calculation correct	$ECL = RHD + \text{Class Levels} + LA$ for all test cases
Stat recalculation accurate	BAB, saves, HP, skills, feats all correct per SRD
Equipment assignment within budget	Total gear value within $\pm 5\%$ of wealth-by-level table
Template generation functional	<code>NPCTemplateDatabase.Generate()</code> returns valid stat blocks for all templates

9.2 Performance Criteria

Criterion	Target
Single NPC generation time	< 5ms
Batch generation (20 NPCs)	< 50ms
Memory: no per-frame allocations from template system	0 GC allocs in steady state

9.3 Compatibility Criteria

Criterion	Measurement
Existing PHB classes unaffected	All Phase1/Phase2/Phase3 class tests pass
Existing NPCDatabase creatures unchanged	Full regression suite passes
Existing combat system works with classed creatures	Combat integration test suite passes